

EOS Quark Engine Service — Complete Technical Documentation

Version: 1.0

Date: May 29, 2026

Original Platform: .NET Framework

Target Platform: Java Spring Boot

Author: Generated from complete .NET codebase analysis

Table of Contents

1. [Executive Summary](#)
 2. [Architecture Overview](#)
 3. [The 7-Step Pipeline](#)
 4. [Task Processing System](#)
 5. [Bloc Model — Document Modification Instructions](#)
 6. [RunTask — Aggregation & Step Building](#)
 7. [QuarkXPress Server Communication \(HTTP\)](#)
 8. [QuarkXPress Server Manager Communication \(SOAP\)](#)
 9. [QXPS_Caller — Step Execution Engine](#)
 10. [QXPS_Modifier — Bloc to XML Transformation](#)
 11. [Dynamic Report Engine \(Process_Dynamique\)](#)
 12. [Document/XML Layer](#)
 13. [Data Formatting \(Data_Type_Helper\)](#)
 14. [Error Handling Strategy](#)
 15. [Configuration](#)
 16. [Example Walkthroughs](#)
 17. [Java Spring Boot Architecture](#)
 18. [Class Reference Index](#)
-

1. Executive Summary

What is EOS Quark?

EOS Quark is a document generation engine used at Société Générale to produce financial reports (fund factsheets, regulatory documents, investment summaries). It takes a Run ID as input and produces professionally designed PDF/QXP/JPG documents by filling pre-designed QuarkXPress templates with live data from an Oracle database.

How it works (one paragraph)

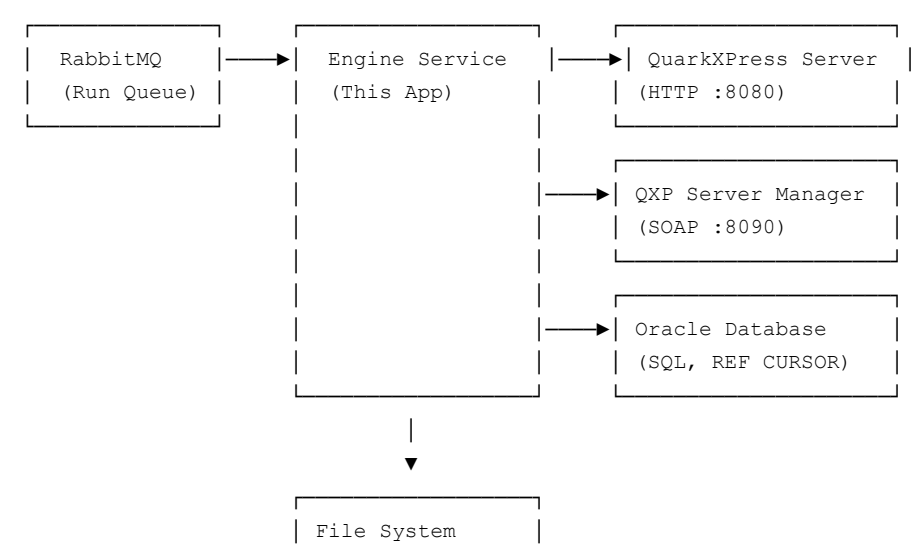
A message arrives on a queue with a Run ID. The engine loads the run configuration from Oracle — which template to use, what SQL queries to run, what tasks to execute. Each task reads data from Oracle and produces document modification instructions (text replacements, page creation, image insertion, table row deletion). These instructions are organized into ordered execution steps and sent to QuarkXPress Server via HTTP and SOAP calls. QuarkXPress Server applies the modifications to the template document and renders the final output as PDF, JPG, and native QXP format. The output files are saved to disk and the run status is persisted to Oracle.

Key technologies

Component	Technology
Original application	.NET Framework (C#)
Database	Oracle (PL/SQL stored procedures, REF CURSOR)
Document engine	QuarkXPress Server (HTTP REST-like API)
Document manager	QuarkXPress Server Manager (SOAP web service)
Message queue	RabbitMQ (in Java rewrite)
Target platform	Java 17+ Spring Boot

2. Architecture Overview

System Context



Internal Architecture

Entry Point: RunMessageListener (RabbitMQ)

|

▼

ProcessRunService.runProcessor(RunIdDto)

|

- ├─ Step 1: Start → RunStartUpdateBusiness → DB update
- ├─ Step 2: Load → GetRunPropertiesBusiness, GetGabaritBusiness,
GetInParamsBusiness, LoadTasksService → DB reads
- ├─ Step 3: Prepare → task.Prepare() for each task
- ├─ Step 4: Process → ProcessTasksService (3-phase loop)
→ TaskProcessStrategy per task type
- ├─ Step 5: ProcessSteps → QXPS_Caller → HTTP/SOAP to QuarkXPress
- ├─ Step 6: Check → Overflow detection, data collection
- ├─ Step 7: Render → QXPS_Caller.Render() → PDF/JPG/QXP output
- └─ Finally: End → Persist status to DB (always runs)

Key Design Principles

Per-task resilience: Individual task failures in Steps 3 and 4 never abort the run. Other tasks proceed independently. Only QuarkXPress Server failures (Step 5) and render failures (Step 7) are fatal.

Mode Degrade: If the template document exceeds 10MB, the engine switches to degraded mode. Steps 3–6 are skipped entirely. Only Step 7 (Render) runs, producing an unmodified PDF. This prevents QuarkXPress Server crashes on oversized documents.

Two output channels: Text updates (simple name=value pairs) go via SOAP to QuarkXPress Server Manager (single round-trip for all updates). Structural modifications (page creation, box cloning, table deletion) go via HTTP to QuarkXPress Server directly (one call per step).

Overflow retry loop: After all modifications are applied, Step 6 checks if any text boxes overflow. If so, affected tasks are reprocessed with alternative "overflow" templates (typically smaller fonts). Steps 4 and 5 re-execute with reduced scope.

3. The 7-Step Pipeline

Pipeline Flow

Launch() receives Run ID

|

▼

Step 1: Start ————— DB: mark "Running"

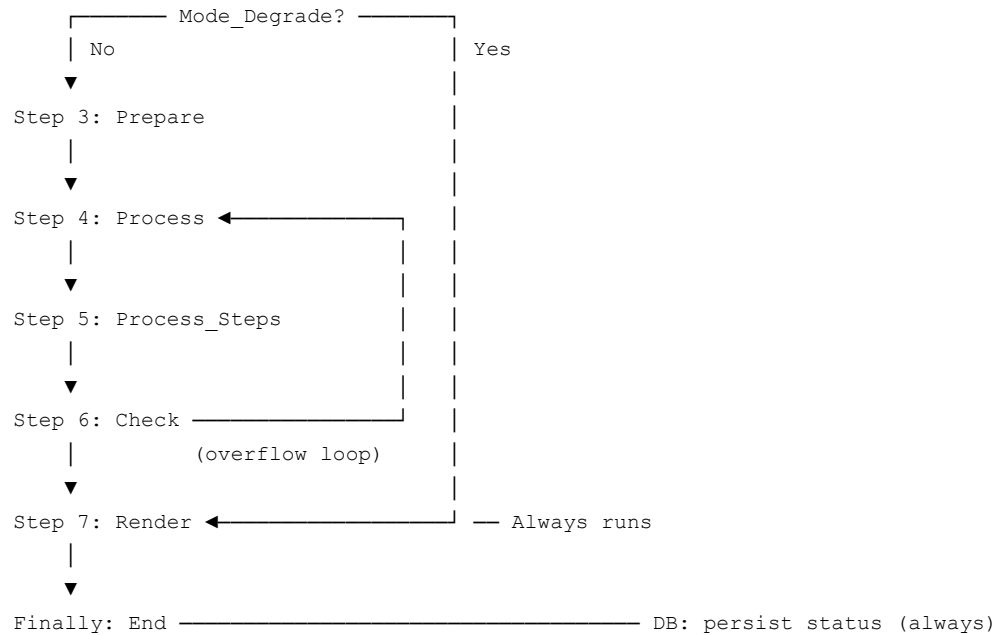
|

▼

Step 2: Load ————— DB: read properties, tasks, params

|

▼



Step 1 — Start

Method: `Launch_Start()` → `Start()`

Purpose: Register the run as "in progress" in Oracle

Behavior: Sets `_status = Run_Status.Running`. The derived `Run` class overrides to write the status to the database.

Error handling: Wrapped in `Universal_Executer<Exception_Run>`. On failure, the run aborts.

Step 2 — Load

Method: `Launch_Load()` → `Load()`

Purpose: Load all run configuration and data from Oracle

Sub-steps (in order):

1. **LoadProperties()** — Reads run configuration: gabarit ID, page count, decimal/group separators, store type, compartment mode, etc.
2. **InitThreadContext()** — Sets the document pool path on `QXPS_File_Manager`, sets thread culture to French with configured separators.
3. **PrepareGabarit()** — Retrieves the QuarkXPress template document (gabarit) and template gabarit from DB or file pool. Checks file size against `Size_Limit_Before_Fail_Soft` to determine `Mode_Degrade`.
4. **LoadInParams()** — Loads input parameters from DB, calls `InParams.InitOraParameters()` to prepare Oracle bind parameters.
5. **LoadTasks()** — Reads task definitions from Oracle via `QXP_PK_RUN.Get_Taches()` stored procedure. Creates task instances using a factory/switch pattern based on task type.
6. **LoadTemplates()** — Loads template definitions for dynamic tasks.

Sub-steps 4–6 are skipped if `Mode_Degrade` is true.

Step 3 — Prepare

Method: `Launch_Prepare()` → `Prepare()`

Purpose: Initialize each task before processing

Behavior: Iterates all tasks with `Todo == true` . Calls `task.Prepare()` on each. What Prepare does depends on task type:

- **Task_Anchor (Dynamique, Compartment):** Loads StartAnchor and EndAnchor positions from template XML. Throws `Exception_Task` if anchors not found.
- **Task_Document:** Determines sub-task type from format (PDF, JPG, DOC, etc.). Sets `ToLoad = true` for file types.
- **Task_SQL/System:** Validates configuration, prepares SQL command.

Error handling: Per-task try/catch. On failure: error added as Critique, task marked `InError = true` , continues to next task. Does NOT abort the run.

Step 4 — Process

Method: `Launch_Process()` → `Process()`

Purpose: Execute business logic for each task to generate document modification instructions (blocs)

Three phases:

Phase 1 — Primary Processing:

```
for each task where Todo == true AND InError == false:
    if task.isModeDegrede():
        add Critique error, skip
    else:
        task.resetProcess() // clears blocsUpdate, blocsModify, dataNamesValues
        task.process()      // generates blocs
    on exception:
        task.setInError(true), add error, continue (do NOT abort)
```

Phase 2 — Secondary Processing (PostProcess):

```
for each task where Todo == true AND InError == false:
    task.postProcess() // only Task_DID uses this
    same error handling — per-task, non-aborting
```

Phase 3 — Verification:

```
for each task where Todo == true AND InError == false:
    if blocsUpdate.isEmpty() AND blocsModify.isEmpty():
        log warning "TaskSansBloc"
    else:
        runTask.add(task) // add to RunTask aggregator
```

Step 5 — Process_Steps

Method: `Launch_Process_Steps()` → `this.Caller.Process()`

Purpose: Send all accumulated document modifications to QuarkXPress Server

Behavior:

1. `QXPS_Caller_Request.Get_Steps()` → `RunTask.Get_Steps()` builds ordered execution steps
2. For each step: call `step.Prepare(ignoreStep)` to evaluate modifier and name values
3. If step has a `Prepare_Step` (pagination dummy pages): execute it first
4. `Execute(step)` — sends modifications via HTTP or SOAP based on `Direct_Call` flag
5. After each step: SaveAs the document, reload for next step
6. Track excluded boxes and partial exclusions

Step 6 — Check

Method: `Launch_Check()` → `Check()`

Purpose: Detect overflow, collect data, trigger reprocessing

Three phases:

Phase 1 — Overflow Control:

```
overflowBoxes = gabarit.XML.GetOverflowBoxes()
for each Task_Dynamique with controlOverflow == true:
    match task.boxNames against overflowBoxes
    if matches found:
        add task to reprocessing list
        task.overflowBoxes = matched boxes

if reprocessing list not empty:
    for all OTHER tasks: set todo = false (except alwaysReprocess == true)
    re-execute Step 4 (Process) and Step 5 (Process_Steps)
```

Phase 2 — SQL Data Collection:

```
if storeDataType includes SQL:
    for each Task_Dynamique and Task_SQL with storeData == true:
        collect dataNamesValues into run.sqlDataNamesValues
```

Phase 3 — Document Data Collection:

```
if storeDataType includes DOCUMENT:
    get all named box values from final document
    filter out auto-named boxes (starting with "Box" or in brackets)
    store in run.docDataNamesValues
```

Step 7 — Render

Method: `Launch_Render()` → `Render()` → `this.Caller.Render()`

Purpose: Generate final document output

IMPORTANT: This step runs EVEN IN Mode_Degrade.

Behavior:

1. Render PDF: Send render message to QuarkXPress Server, receive PDF bytes
2. Render QXP: SaveAs with new name, retrieve file from pool

3. Render JPG: Send JPEG render message, receive JPG bytes
4. Store results in `Run_Result` (`Final_PDF`, `Final_QXP`, `Final_JPG`)
5. `Save_Result()` : Write files to disk if configured

Finally Block (Always Executes)

```

_step = Run_Step.End
_finGeneration = DateTime.Now
audit.End()
try:
    End() → Run.End() → Proxy_Run.End_Run()
    → BEGIN TRANSACTION
    → UPDATE run status (GENERATED or ERROR)
    → INSERT documents (PDF, QXP, JPG) into DB
    → COMMIT
catch:
    log error (don't re-throw – cleanup must complete)

```

4. Task Processing System

Task Type Hierarchy

```

TaskBase (abstract)
├─ Task_Anchor (abstract) – adds StartAnchor/EndAnchor
│   └─ Task_Dynamique – SQL-driven dynamic reports
│       └─ Task_Compartiment – nested sub-document runs
├─ Task_SQL – direct SQL queries → text replacements
├─ Task_System – simple value/box operations
│   └─ Task_DID – Document Identity (PostProcess)
├─ Task_Document – image/PDF/document insertion
└─ Task_QXP_Previous – content from previous document versions

```

Task Type Summary

Task Type	Process Logic	BlocsUpdate	BlocsModify	Uses PostProcess	External Dependencies
Task_SQL	Execute SQL, map columns to boxes	Text values per column	Table/row removal (exceptions)	No	Oracle DB
Task_Dynamique	SQL → DReport → geometry → blocs	(via BlocsModify)	Pages, positioned boxes, anchors	No	Oracle DB, Template
Task_System	Simple value or box operation	Text value (Value mode)	Box repositioning (Box mode)	No	None

Task_DID	Detect pagination, update identity	DID value (NameValue mode)	DID repositioning (Modifier mode)	Yes	Cross-task scan
Task_Compartiment	Run child documents, merge output	None	Pages, cloned boxes from children	No	Child runs, DB
Task_Document	Insert image/PDF/document file	Previous QXP values	Image boxes, PDF pages	No	File system

TaskBase Properties

Property	Type	Default	Purpose
id	int	—	Task identifier
todo	boolean	true	Whether task should execute
inError	boolean	false	Set on failure, excludes from further processing
alwaysReprocess	boolean	false	Survives overflow re-execution
modeDegrede	boolean	false	Task-level degraded mode
controlOverflow	boolean	false	Whether overflow detection applies
destinationBlocName	String	""	Target box/anchor name in document
sourceBlocName	String	""	Source box name (for copy tasks)
subTaskType	SubTaskType	—	Detailed sub-type
blocsUpdate	Map	LinkedHashMap	Text content replacements
blocsModify	Map	LinkedHashMap	Structural modifications
dataNamesValues	List	—	Collected data for audit storage
debugInfo	String	—	"[id - commentaire]" for logging

Process_SQL — Detailed Flow

```

Process(Task_SQL task):
    1. Clone Oracle parameters from run.inParams
    2. Execute task.SQL via Proxy_Generic.GetReader()
    3. For each row:
        a. Read NOM_BLOC (box name) and VALEUR_BLOC (value)
        b. If Integer_N1 == false AND name ends with "N1" → skip
        c. Format value via DataTypeHelper.OutputToString()

```



```

    d. Create BlocBox(task, name, formattedValue), action = UPDATE
    e. Check for duplicates → add error if duplicate, skip
    f. Add to task.blocsUpdate
    g. If storeData: add to task.dataNamesValues
4. Check_Task_Exception(task):
    For each exception rule:
        If monitored bloc is empty/null/not found:
            If type == LINE: create BlocLigne(REMOVE) per row index
            If type == TABLE: create BlocTable(REMOVE)
            Add to task.blocsModify

```

Process_Dynamique — Detailed Flow

```

Process(Task_Dynamique task):
    Precondition: task.run.gabaritTemplate must not be null

    Stage 1 — Get_Report(prp, reader):
        Execute SQL, build DReport hierarchy from result rows
        Each row contains: ID_SECTION, ID_TABLE, ID_GROUPE, ID_LIGNE,
                           NOM_BLOC, VALEUR_BLOC, TEMPLATE, ROW_LEVEL

    Stage 2 — Check_Report(prp, report):
        For each DCell: resolve TElement from template
        Remove cells/rows/tables/sections with no TElement

    Stage 3 — Prepare_Report(prp, report):
        Calculate geometry for all cells
        Handle page breaks, column breaks, multi-column layout
        Track cursor position, manage anchor boundaries
        Populate task.boxNames for overflow detection

    Stage 4 — Render_Boxes(prp, report):
        Create BlocPage(CREATE) for new pages
        Create BlocPage(REMOVE) for old pages between anchors
        Move start/end anchors (Pagination = true)
        Convert each DCell to BlocBox via TElement_Helper.Get_Bloc()
        Add all to task.blocsModify

```

Task_DID — PostProcess Logic

```

PostProcess():
    1. Scan ALL tasks for any bloc with Pagination == true
       → modifier = true if found

    2. Calculate new DID value via Document_Identity_Helper

    3a. If modifier (pagination changed):
        - Get DID box position from template
        - Clone MOVE_BLOC_VALUE template element
        - Set position, UID, and new DID value
        - Set TBox_Action = MOVE, TBox_Pagination = true

    3b. If not modifier (pagination unchanged):
        - Verify DID box exists in template

```

- Set Value = new DID value
4. Delegate to Process_System.Process(this)
 - Creates BlocBox in blocsUpdate (NameValue) or blocsModify (Modifier)

Process_Compartment — Child Run Flow

```
Process(Task_Compartment task):
  1. Prepare_Runs():
    - Query DB for child run IDs associated with this compartment
    - Create Run (generate) or Run_Previous (retrieve cached) instances

  2. Execute_Runs():
    - Call child.Launch() for each child run
    - Collect trace logs from child into parent

  3. Render_Runs():
    If Incorporate mode:
      - For each child run: extract QXP project boxes
      - Rename boxes with suffix (ID_Fnd_Code) to prevent conflicts
      - Group boxes by page, create BlocPage(CREATE) per page
      - Remove old pages between anchors
      - Move start/end anchors with Pagination = true
    Else:
      - Set task.todo = false (nothing to incorporate)

  4. Cleanup: Clear child runs, force GC
```

Process_Document — Image/PDF Handling

```
Process(Task_Document task):
  Validate destinationBlocName exists

  Switch on subTaskType:
    File_IMG (JPG, GIF, EPS, TIF):
      - Get IMG template from TElement_Helper
      - Set content.value = absolute file path
      - Optional 90° rotation
      - Create BlocBox → blocsModify

    File_PDF:
      - For each PDF page:
        If more boxes than pages → remove extra boxes
        If more pages than boxes → create new boxes
      - Get template per box name
      - Set content.value = PDF file path
      - Apply position for pages after first
      - Optional rotation + offset
      - Each BlocBox → blocsModify

    File_DOC/RTF/XTG:
      - Get RTF_DOC_XTG template
      - Set content.value = "file:{path}"
      - Create BlocBox → blocsModify
```

```
File_QXP_Data:
- Split source/destination names by '|'
- For each pair: copy value (or full style) from source to destination
- BlocBox → blocsUpdate (value only) or blocsModify (with style)
```

5. Bloc Model — Document Modification Instructions

Bloc Hierarchy

```
BlocBase (abstract)
├─ BlocBox — single box (text value OR structural TBox)
├─ BlocPage — page creation/removal (Pagination always true)
├─ BlocTable — table removal
├─ BlocLigne — specific row removal within a table
└─ BlocGroup — multiple related boxes as one unit
```

BlocBase Properties

Property	Type	Default	Purpose
name	String	—	Box/table/page identifier in document
action	BlocAction	NONE	Operation: NONE, UPDATE, CREATE, REMOVE, MOVE
task	TaskBase	—	Parent task that created this bloc
relativePage	int	0	Page offset from task's destination page
pageId	int	MIN_VALUE	Absolute page ID (calculated)
pageIdOriginale	int	MIN_VALUE	Original page ID (immutable during repagination)
spreadId	int	(calculated)	Spread containing this page
condName	String	""	Conditional name for page evaluation
parentName	String	""	Parent element (table/group)
pagination	boolean	false	Does this bloc affect document pagination?
lagSpread	boolean	false	Spread offset for double-page layouts
indexMaquette	int	4	Master page template index
nbBox	int	1 (virtual)	Number of boxes in this bloc
exclude	boolean	false	Excluded due to box count limits

BlocBox — Two Modes

Value Mode (for BlocsUpdate):

```
BlocBox bloc = new BlocBox(task, "CustomerName", "John Smith");
bloc.setAction(BlocAction.UPDATE);
// bloc.getValue() = "John Smith"
// bloc.getSrcBox() = null
```

TBox Mode (for BlocsModify):

```
BlocBox bloc = new BlocBox(task, "BoxName", tbox);
bloc.setAction(BlocAction.CREATE);
// bloc.getValue() = null
// bloc.getSrcBox() = QXPSMSDK.Box with geometry, content, etc.
// bloc.getSrcExtraBox() = optional secondary box (for PDF offset)
```

BlocPage — Page Operations

```
// Create new page
BlocPage newPage = new BlocPage(task, "T1_P2");
newPage.setAction(BlocAction.CREATE);
newPage.setRelativePage(2);
// Pagination automatically set to true in constructor

// Remove old page
BlocPage oldPage = new BlocPage(task, "T1_P5");
oldPage.setAction(BlocAction.REMOVE);
// getNbBox() returns actual box count on page (from template info)
```

BlocsUpdate vs BlocsModify

Aspect	BlocsUpdate	BlocsModify
Command type	NameValue (simple text replacement)	Modify (structural XML command)
Bloc types	BlocBox with Value string only	BlocBox with SrcBox, BlocPage, BlocTable, BlocLigne
Actions	Implicitly UPDATE	CREATE, REMOVE, MOVE
Producers	Process_SQL, Process_System (value mode)	Process_Dynamique, Process_SQL (exceptions), Process_System (box mode), Process_Compartment, Process_Document
Effect	Replace text in existing boxes	Create/delete pages, position boxes, remove tables/rows
Sent via	SOAP (Direct_Call = false)	HTTP (Direct_Call = true)

6. RunTask — Aggregation & Step Building

Purpose

`RunTask` collects all blocs from all processed tasks and organizes them into an ordered list of `RunTaskStep` objects for sequential execution against QuarkXPress Server.

RunTask.Add(task)

```
if task.blocsModify.count > 0 → add to _tasks_Modifier
if task.blocsUpdate.count > 0 → add to _tasks_Update
```

RunTask.Get_Steps() — Step Building Algorithm

```
Phase 1: UPDATE STEP
    Create single RunTaskStep (Direct_Call = false)
    Collect ALL blocsUpdate from ALL _tasks_Update
    → Step[0] if any updates exist

Phase 2: PAGINATION MODIFY STEP
    Create RunTaskStep for pagination blocs
    For each task in _tasks_Modifier:
        For each bloc in task.blocsModify:
            If bloc.pagination == true → add to pagination step
            Else → add to normal modify step (track nb_Modify count)

            If nb_Modify > Step_Limit (e.g., 5000):
                Finalize current modify step → add to _steps_temp
                Create new modify step, reset counter

Phase 3: FINAL ORDERING
    [0] UPDATE step (NameValues, Direct_Call=false)
    [1] PAGINATION step (if any pagination blocs)
    [2..N-1] OVERFLOW SPLIT steps (from step splitting)
    [N] FINAL MODIFY step (remaining structural blocs)

Number each step with sequential index.
```

Step Limit — Preventing QuarkXPress Crashes

When more than `Step_Limit` (default: 500) structural modifications accumulate in a single step, the step is split into multiple smaller steps. This prevents QuarkXPress Server from crashing with too many simultaneous modifications.

Example:

```
Task_Dynamique produces 1200 BlocBox modifications
Step_Limit = 500
```

Result:

```
Step[0]: UPDATE (all text values)
Step[1]: PAGINATION (page create/delete)
Step[2]: MODIFY_1 (blocs 1-500)
```

```
Step[3]: MODIFY_2 (blocs 501-1000)
Step[4]: MODIFY_3 (blocs 1001-1200)
```

RunTaskStep.Prepare(ignoreStep)

1. Evaluate_Info() – delegate callback to tasks for latest spreadID/layout info
2. Evaluate_Modifier_And_Values():
For each bloc in blocsUpdate → create NameValueParam (box name → value)
For each bloc in blocsModify → prepare for structural modification
3. Evaluate_Limitation(ignoreStep) – handle box count limits, mark excluded blocs
4. _modifier.AddRange(blocsModify) – add blocs to QXPS_Modifier hierarchy

7. QuarkXPress Server Communication (HTTP)

Overview

QuarkXPress Server exposes a REST-like HTTP API on port 8080. Document operations are encoded in URL paths and query parameters.

HTTP Operations

Operation	Method	URL Pattern	Body	Response
SDKCall (most operations)	GET	{base}/{paths}/{doc}? {queries}	None	Text or binary
SDKCall (modify)	POST	{base}/{paths}/{doc}? {queries}	Multipart form-data	Text or binary
AddFile	PUT	{base}/addfile/{poolPath}	Raw bytes	Discarded
GetFileData	GET	{base}/getfile/{poolPath}	None	byte[]
DeleteFile	DELETE	{base}/deletefile/{poolPath}	None	Discarded

All use HTTP 1.0 with 1-hour timeout (3600000 ms).

Message Classes

Each message type knows its own URL path segment, query parameters, priority, and whether it requires POST.

Message Class	getPath()	getQuery()	isPost	Priority
ParamsValueMessage	null	name1=val1&name2=val2	false	BELOW_NORMAL
ModifyMessage	null	modify=file:{filename}	false	NORMAL
SaveAsMessage	"saveas"	newname=X&replace=false&...	false	BELOW_NORMAL

DeleteMessage	"delete"	null	true	LOWEST
AddFileMessage	"addfile"	null	true	LOWEST
JpegMessage	"jpeg"	page=X&spread=Y&box=Z	false	ABOVE_NORMAL
PdfMessage	"pdf"	colorimagedownsample=X&...	false	ABOVE_NORMAL
QxpMessage	"qxpdoc"	(empty)	false	ABOVE_NORMAL
XmlMessage	"xml"	box=X	false	LOWEST
LiteralMessage	"literal"	null	false	BELOW_NORMAL

Priority Enum

```

LOWEST = 1
BELOW_NORMAL = 2
NORMAL = 3
ABOVE_NORMAL = 4
HIGHEST = 5

```

Higher priority messages appear earlier in the URL path (sorted descending).

URL Construction (QxpsRequestBuilder)

```

Input: List<Message>, documentName
Process:
  1. Sort messages by priority (descending)
  2. Collect getPath() from each → filter nulls → join with "/"
  3. Append documentName as final path segment
  4. Collect getQuery() from each → filter nulls → join with "&"
  5. Build URI: baseUrl + "/" + path + "?" + query
  6. Detect POST: at most one message may have isPost=true
  7. Return QxpsRequestInfo(uri, isPost, postData)

```

Multipart POST Format (for modify operations with binary data)

```

--{guid-boundary}
Content-Disposition: form-data; name="file"; filename="fileData.bin"
Content-Type: binary/octet-stream

{binary data bytes}
--{guid-boundary}--

```

Response Parsing

```

Content-Type "text/plain", "text/xml", "text/html", "" → read as String
Everything else → read as byte[] (PDF, JPG, QXP data)

```

8. QuarkXPress Server Manager Communication (SOAP)

Overview

QuarkXPress Server Manager exposes a SOAP web service for higher-level document operations. Multiple operations can be chained into a single SOAP request for efficiency.

SOAP Endpoint

URL: `http://srvcldqxp001.dns43.socgen:8090/qxpsm/services/RequestService`
Namespace: `http://webservice.manager.quark.com`

Request Chaining Pattern

Requests form a linked list. Each request's `.request` field points to the next:

```
RequestParameters
  .request → ModifierRequest
    .request → QuarkXPressRenderRequest
      .request → SaveAsRequest
        .request → null
```

All requests are sent in one SOAP message. The server processes them sequentially.

QXPSM Request Types

Request Class	SDK Type	Purpose
QxpsmRequestParamsValue	RequestParameters	Update box text values
QxpsmRequestModifier	ModifierRequest	Structural document changes
QxpsmRequestSaveAs	SaveAsRequest	Save document to pool
QxpsmRequestAddFile	AddFileRequest	Upload file to pool
QxpsmRequestQxp	QuarkXPressRenderRequest	Render as QXP
QxpsmRequestPdf	PDFRenderRequest	Render as PDF
QxpsmRequestJpeg	JPEGRenderRequest	Render as JPEG
QxpsmRequestXml	DeconstructRequest	Export as XML

Two Modes

Autonomous mode: Constructor takes `documentName`, calls `SDKCall()` immediately. Used for standalone operations like `GetProject()`.

Chained mode: Constructor takes no `documentName`. Request is added to `QxpsmCall.requests` list. `SDKCall()` sends all chained requests together.

9. QXPS_Caller — Step Execution Engine

Execute(step) — Decision Logic

```
For each RunTaskStep:

    1. VALUE UPDATES (step.nameValues)
        IF step.directCall:
            → Create QXPS_Message_ParamsValue → add to HTTP messages
        ELSE:
            → Create QXPSM_Request_ParamsValue → add to SOAP chain

    2. STRUCTURAL MODIFICATIONS (step.modifier not empty)
        IF step.directCall:
            → Serialize modifier to XML → upload to pool
            → Create QXPS_Message_Modify(filename) → add to HTTP messages
        ELSE:
            → Create QXPSM_Request_Modifier(project) → add to SOAP chain

    3. RENDER QXP
        → Create render message/request → add to messages/chain

    4. SAVE DOCUMENT
        → Create SaveAs message/request → add to messages/chain

    5. EXECUTE
        IF step.directCall:
            → QXPS_Caller.SDKCall() (HTTP)
        ELSE:
            → QXPSM_Caller.SDKCall() (SOAP - single round trip)

    6. UPDATE CONTEXT
        → Update document name for next step
        → Increment execution counter
```

Direct Call vs SOAP — When Each Is Used

Step Type	Direct_Call	Protocol	Reason
UPDATE (text values)	false	SOAP	Efficient: all values in one request
PAGINATION	true	HTTP	Page operations need direct server control
MODIFY (structural)	true	HTTP	Complex XML modifications

10. QXPS_Modifier — Bloc to XML Transformation

Purpose

Converts a flat list of BlocBase objects into a hierarchical QXPSMSDK.Project XML structure that QuarkXPress Server understands.

Internal Hierarchy

```
Projet (internal)
└─ Layout (keyed by Layout_ID)
    └─ Spread (keyed by Spread_ID)
        ├── Boxes (keyed by name)
        ├── Boxes_Extra (keyed by name – for SrcExtraBox)
        ├── Groups (keyed by name)
        ├── Tables (keyed by name)
        │   └─ Lignes (keyed by row index)
        └─ Pages (keyed by UID)
```

Bloc Type Dispatch

```
Add(BlocBase bloc):
    if bloc.exclude → skip
    if bloc is BlocBox → Add(BlocBox)           → Spread.Boxes
    if bloc is BlocGroup → Add(BlocGroup)       → Spread.Groups
    if bloc is BlocTable → Add(BlocTable)       → Spread.Tables
    if bloc is BlocLigne → Add(BlocLigne)       → Table.Lignes
    if bloc is BlocPage → Add(BlocPage)        → Spread.Pages
```

GetProject() → QXPSMSDK.Project

Converts internal hierarchy to SDK objects:

Internal	SDK Output	Operation Mapping
Box (UPDATE)	QXPSMSDK.Box	operation = "UPDATE"
Box (CREATE)	QXPSMSDK.Box	operation = "CREATE", UID = ""
Box (MOVE)	QXPSMSDK.Box	operation = "UPDATE" + geometry
Group (CREATE)	Multiple QXPSMSDK.Box	Each box: operation = "CREATE"
Table (REMOVE)	QXPSMSDK.Table	operation = "DELETE"
Table (lines)	QXPSMSDK.Table	operation = "UPDATE", rows with DELETE
Page (CREATE)	QXPSMSDK.Page	operation = "CREATE"
Page (REMOVE)	QXPSMSDK.Page	operation = "DELETE"

GetIOValue() — XML Serialization

```
GetProject() → QXPSMSDK.Project
    → QXP_Project_Serializer.Transform(project)
    → XmlWriter serialization
    → UTF-8 bytes (no BOM)
    → IOValue for HTTP transmission
```


└─ Info (DZone with calculated geometry)

Stage 1 — Get_Report (SQL → DReport)

SQL result columns drive hierarchy creation:

Column	Change Triggers
ID_SECTION	New DSection created
ID_TABLE	New DTable created in current section
ID_GROUPE	New DCell created (template change)
ID_LIGNE	New DRow created in current table
TEMPLATE	Template lookup in run.templates
NOM_BLOC	Added to cell.newNames
VALEUR_BLOC	Added to cell.values
ROW_LEVEL	(optional) Sets row.level for break rules

Template resolution: If TEMPLATE name exists in `run.templates` → use it. If not → create temporary template with `srcName = templateName`.

Absolute vs Relative: If `template.absolute == true` → cell added to `section.cells` (absolute positioning). Otherwise → cell added to row in table (relative positioning).

Stage 2 — Check_Report (Template Validation)

```
For each section:
  For each cell (absolute and relative):
    Determine overflow flag: cell contains overflow box names?
    Resolve TElement: TElement_Helper.Evaluate_TElement(task, cell)
      → Lookup template.srcName (or srcNameOverflow) in QXP_Project.Elements
    If TElement is null → mark cell for removal

  Remove invalid cells → empty rows → empty tables → empty sections
```

Stage 3 — Prepare_Report (Geometry Calculation)

This is the most complex stage. It calculates where every cell goes in the final document.

Key concepts:

- **Anchor boundaries:** Available height = EndAnchor.Top - StartAnchor.Bottom
- **Cursor:** Tracks current position (left, top) as cells are placed
- **Page breaks:** When accumulated row height exceeds available height → new page
- **Column breaks:** When multi-column layout, overflow goes to next column first

- **Break rows:** Rows with `page_break` or `column_break` flag are repeated on each new page/column

Algorithm:

```

    available_height = endAnchor.top - startAnchor.bottom
    cursor = DPoint(startAnchor.right, startAnchor.bottom)
    page = 1

    For each section:
        Process absolute cells (repeat on Other_Page, Last_Page, etc.)

    For each table:
        For each row:
            If row.page_break or row.column_break:
                Store as header row for repetition
                Mark for removal from final output
                Continue

        Calculate cell geometry via Prepare_Report_Cells()
        Add row height to table height

        If tableHeight > available_height:
            If column < nb_Column:
                New column (increment column, add column_break rows)
            Else:
                New page (increment page, add page_break rows, reset)
                Add absolute cells to new page

        Update cursor position

    Remove break rows from table

    Store all cell box names in task.boxNames (for overflow detection)
    Set report.info.nb_page = total pages

```

Stage 4 — Render_Boxes (DReport → Blocs)

1. Create BlocPage(CREATE) for each new page (0 to nb_page-1)
2. Create BlocPage(REMOVE) for each old page between anchors
3. Move start anchor to page 0 (Pagination = true)
4. Move end anchor to last page (Pagination = true)
5. For each DCell (absolute and relative):
 - BlocBase bloc = TElement_Helper.Get_Bloc(cell, task)
 - Clone TElement, update name/value/position
 - Return BlocBox or BlocGroup
 - Add to task.blocsModify

Break Rules Format

Rules follow pattern `x:y` separated by `|` :

- **X** = row level(s) that trigger a break
- **Y** = row level(s) to repeat when break occurs

Examples:

Rule	Meaning
1:0-1	When level 1 changes page, repeat levels 0 and 1
2-4:1-3	When levels 2/3/4 change page, repeat levels 1/2/3
5:L2	When level 5 changes, bring 2 rows above (by count, not level)

12. Document/XML Layer

QXP_XML — Document Structure Analysis

XPath-based queries on the QuarkXPress document XML structure.

Method	XPath Pattern	Purpose
GetValue (blocName)	//ID[@NAME='{0}']/../PARAGRAPH/RICHTEXT/text()	Get box text content
GetUID (blocName)	//ID[@NAME='{0}']/@UID	Get unique identifier
GetPageNum (blocName)	(//BOX[ID[@NAME='{0}']] //TABLE[...]) /GEOMETRY/@PAGE	Get page number
GetBlocInfo (blocName)	Position + UID + Page	Get full geometry info
GetNbLignes (tableName)	//TABLE[ID[@NAME='{0}']]//ROW	Count rows in table
GetOverflowBoxes ()	//OVERMATTER/../../../../ID/@NAME	Get overflowing box names
GetNamesValuesBoxes ()	All CT_TEXT box names + RICHTEXT values	Get all named values
GetListBoxNameStartWith (name)	//ID[starts-with(@NAME,'{0}')]/@NAME	Find boxes by prefix

QXP_Project — Template Element Indexing

```
Analyse(task, logError):
    Walk: Project → Layouts → Spreads → Boxes/Tables/Groups
    For each box: Create TBox wrapper, add to Elements dictionary, call Evaluate()
    For each table: Create TTable wrapper, add to Elements dictionary
    For each group: Collect first (groups reference boxes by name)
    Process groups after all boxes indexed
    Set analysed = true

Elements dictionary: Dictionary<String, TElement>
```

Key: element name (e.g., "TitleBox", "DataTable")
Value: TElement instance (TBox, TTable, TGroup)

Document Class

Wraps binary document data with format information:

```
Document:
    id: int
    data: byte[]
    type: Document_Type (PDF, IMG, QXP, XTG, DOC, RTF)
    format: String
    fileName: String (prefix + id + format)
    filePoolPath: String (relative pool path)
    fileFullPath: String (absolute file path)
    mode_degrade: boolean (true if data.length > SizeLimitBeforeFailSoft)
    xml: QXP_XML (lazy-loaded, cached)
    qxpProject: QXP_Project (lazy-loaded via SOAP getProject, cached)
```

13. Data Formatting (Data_Type_Helper)

Supported Data Types

Enum	Value	Formatting
Unspecified	0	Treated as Text
Text	1	Pass-through (null → nullString)
Int	2	Integer (showZero controls zero display)
Decimal	3	#,##0.00 or #,##0.## pattern
Currency	4	Decimal + currency symbol
Date	5	Culture-specific date pattern
DateTime	6	Culture-specific datetime pattern
Pourcentage	7	Decimal + " %"
Boolean	8	Boolean formatting

Decimal Formatting

decimal_significative = true: Trailing zeros shown (#,##0.00 → "1,500.00")

decimal_significative = false: Trailing zeros hidden (#,##0.## → "1,500.5")

Null/Zero Handling

```
if value is null or empty → return nullString (default: " ")
```

```
if value is 0 and showZero == false → return nullString
if value is 0 and showZero == true → return formatted "0" or "0.00"
```

Cultural Settings

Thread culture is set to French (FR) with configurable decimal and group separators. The `Data_Type_Helper` constructor takes `Run_Properties` which provides the separator configuration and date patterns.

14. Error Handling Strategy

Error Types

Error_Type	Value	Meaning
Unspecified	0	Default/unknown
Bloquante	1	Blocking — prevents further processing
Critique	2	Critical — serious but may continue
Avertissement	3	Warning — non-critical
Information	4	Informational — diagnostic

Execution_Helper — Generic Exception Wrapping

```
// Single type: wraps any exception in T_Exception
Universal_Executer<Exception_Run>(() -> {
    doSomething();
});
// If doSomething() throws → new Exception_Run(originalException) is thrown

// Triple type: routes exceptions by type
Universal_Executer<Exception_Run, QXPS_Exception, Exception_Render>(() -> {
    callQuarkServer();
});
// If QXPS_Exception thrown → new Exception_Render(original) thrown
// If any other exception → new Exception_Run(original) thrown
```

Error Handling by Step

Step	Task Error	System Error
Step 1 (Start)	—	Abort run
Step 2 (Load)	—	Abort run
Step 3 (Prepare)	Mark task InError, continue	Abort run
Step 4 (Process)	Mark task InError, continue	Abort run

Step 5 (Process_Steps)	—	Abort run
Step 6 (Check)	—	Abort run
Step 7 (Render)	—	Abort run

Error Collection

```
run.errors.add(Error_Type.Critique, "Message: {0}", arg1);
// Creates Error object with formatted message
// Try/catch protected – error logging itself never throws
// Errors accumulate in BaseList<Error> collection
// ToString() formats all errors for logging
```

15. Configuration

EngineCoreSetting

Setting	Default	Purpose
Step_Limit	500	Max blocs per execution step (prevents QXP crashes)
Size_Limit_Before_Fail_Soft	10,485,760 (10MB)	Template size threshold for Mode_Degrade
Average_Box_Size	25.0 KB	Box size estimate for memory calculations
Max_Boxes_Per_Page	1000	Safety limit per page
Max_Pages_Per_Document	5000	Safety limit for total pages
Default_Timeout_Seconds	3600 (1 hour)	Task execution timeout
Max_Retry_Count	3	Failed operation retries
Auto_Degradation_Enabled	true	Allow automatic Mode_Degrade

Application Configuration (application.yml)

```
qxps:
  server:
    url: "http://srvcldvapd001.dns43.socgen:8080"
    timeout: 3600000

qxpsm:
  soap:
    endpoint: http://srvcldqxpu001.dns43.socgen:8090/qxpsm/services/RequestService
    namespace: http://webservice.manager.quark.com
    connection-timeout: 10000
    timeout: 30000

documentpool:
```

basePath: D:\Documents

16. Example Walkthroughs

Example 1 — Simple SQL Task (Happy Path)

Scenario: Run ID 1001 generates a fund factsheet. Task_SQL #10 fills in the fund name and NAV value.

Step 1 — Start:

```
DB: UPDATE QXP_RUN SET STATUS = 'RUNNING' WHERE ID_RUN = 1001
```

Step 2 — Load:

```
DB: SELECT * FROM QXP_PK_RUN.Get_Properties(1001) → RunProperties
    gabaritId = 50, idLangue = 1, dateEcheance = 2026-01-31
    storeType = SQL, typeRapport = FONDS
```

```
DB: Get gabarit document ID=50 → byte[245000] (QXP file)
    Upload to pool: PUT http://server:8080/addfile/R_1001/G_50.qxp
```

```
DB: Get InParams → { ":ID_FUND" = "FR0001234", ":DATE" = "2026-01-31" }
    InitOraParameters() → OraParameter[] ready for SQL
```

```
DB: Get_Taches(1001) → [
    { ID_TACHE=10, TYPE=SQL, SQL="SELECT NOM_BLOC, VALEUR_BLOC FROM V_FUND_DATA WHERE ID = :ID_FUND"
      DATA_TYPE=3, NB_DECIMAL=2, SHOW_ZERO=true, STORE_DATA=true }
]
    Create Task_SQL(10, run)
```

Step 3 — Prepare:

```
Task_SQL #10: Prepare() → validate SQL, check DB connection
```

Step 4 — Process:

```
Task_SQL #10: Process()
    Clone parameters: [:ID_FUND = "FR0001234", :DATE = "2026-01-31"]
    Execute SQL: SELECT NOM_BLOC, VALEUR_BLOC FROM V_FUND_DATA WHERE ID = :ID_FUND

    Row 1: NOM_BLOC = "FundName", VALEUR_BLOC = "Euro Growth Fund"
        → DataTypeHelper.OutputToString("Euro Growth Fund", Text, ...) = "Euro Growth Fund"
        → BlocBox("FundName", "Euro Growth Fund"), action = UPDATE
        → task.blocsUpdate["FundName"] = bloc

    Row 2: NOM_BLOC = "NAV_Value", VALEUR_BLOC = 125.4567
        → DataTypeHelper.OutputToString(125.4567, Decimal, 2, true, " ", true) = "125.46"
        → BlocBox("NAV_Value", "125.46"), action = UPDATE
        → task.blocsUpdate["NAV_Value"] = bloc

    Row 3: NOM_BLOC = "TotalAssets", VALEUR_BLOC = 1500000.00
```

```

→ DataTypeHelper.OutputToString(1500000.00, Decimal, 0, true, " ", true) = "1 500 000"
→ BlocBox("TotalAssets", "1 500 000"), action = UPDATE
→ task.blocsUpdate["TotalAssets"] = bloc

```

DataNamesValues stored: [("FundName","Euro Growth Fund"), ("NAV_Value","125.46"), ("TotalAssets"

```

Phase 2 (PostProcess): Task_DID runs
  Scans all tasks → no pagination blocs found → NameValue mode
  DID value = "ABCD1234-20260131-001"
  BlocBox("DID", "ABCD1234-20260131-001"), action = UPDATE
  → task.blocsUpdate["DID"] = bloc

```

```

Phase 3 (Verify):
  Task_SQL #10: 3 blocsUpdate → add to RunTask
  Task_DID: 1 blocUpdate → add to RunTask

```

Step 5 — Process_Steps:

```

RunTask.Get_Steps():
  Step[0]: UPDATE (Direct_Call = false)
    NameValues = [
      ("FundName", "Euro Growth Fund"),
      ("NAV_Value", "125.46"),
      ("TotalAssets", "1 500 000"),
      ("DID", "ABCD1234-20260131-001")
    ]

QXPS_Caller.Execute(step[0]):
  Direct_Call = false → SOAP path
  Create QXPSM_Request_ParamsValue(nameValues)
  Create QXPSM_Request_QXP()
  Create QXPSM_Request_SaveAs("/pool/R_1001/", "G_50_2.qxp", true, false)

Chain: ParamsValue → QXP → SaveAs
QXPSM_Call.SDKCall() → single SOAP POST to :8090

QuarkXPress Manager processes:
  1. Update FundName = "Euro Growth Fund" in document
  2. Update NAV_Value = "125.46" in document
  3. Update TotalAssets = "1 500 000" in document
  4. Update DID = "ABCD1234-20260131-001" in document
  5. Render QXP
  6. Save as G_50_2.qxp

Update document name to G_50_2.qxp for next step

```

Step 6 — Check:

```

GetOverflowBoxes() → [] (no overflow)
SQL data collection → store dataNamesValues to run.sqlDataNamesValues
No reprocessing needed

```

Step 7 — Render:

Render PDF:

```
QXPS_Message_PDF(colorCompression="medium")
SDKCall() → GET http://server:8080/pdf/R_1001/G_50_2.qxp?colorcompression=medium
Response: byte[54000] (PDF data)
→ Document("FinalDoc", "PDF", byte[54000])
```

Render QXP (SaveAs):

```
QXPS_Message_SaveAs(newName="G_50_3.qxp")
SDKCall() → GET http://server:8080/saveas/R_1001/G_50_2.qxp?newname=G_50_3.qxp
GetFileData("R_1001/G_50_3.qxp") → byte[250000]
→ Document("FinalDoc", "QXP", byte[250000])
```

Save_Result():

```
Save PDF to D:\Documents\R_1001\DF_1001.pdf
Save QXP to D:\Documents\R_1001\DF_1001.qxp
```

Finally:

```
status = GENERATED
finGeneration = 2026-01-31T14:32:15
End() → Proxy_Run.End_Run():
BEGIN TRANSACTION
UPDATE QXP_RUN SET STATUS='GENERATED', DATE_FIN='2026-01-31 14:32:15' WHERE ID_RUN=1001
INSERT INTO QXP_DOCUMENT(ID_RUN, FORMAT, PATH, SIZE) VALUES (1001, 'PDF', '...', 54000)
INSERT INTO QXP_DOCUMENT(ID_RUN, FORMAT, PATH, SIZE) VALUES (1001, 'QXP', '...', 250000)
COMMIT
```

Example 2 — Dynamic Task with Overflow

Scenario: Run ID 2001 generates a detailed holdings report. Task_Dynamique #20 creates a multi-page table that overflows on first pass.

Step 2 — Load (relevant parts):

```
Task_Dynamique #20 loaded:
SQL = "SELECT ID_SECTION, ID_TABLE, ID_GROUPE, ID_LIGNE, NOM_BLOC, VALEUR_BLOC,
        TEMPLATE, ROW_LEVEL FROM V_HOLDINGS WHERE FUND_ID = :ID_FUND ORDER BY ..."
destinationBlocName = "Ancre_Holdings"
controlOverflow = true
storeData = true
pageBreakRules = "1:0-1" (when level 1 changes page, repeat header rows 0-1)
nbColumn = 1
```

Step 3 — Prepare:

```
Task_Dynamique #20: Prepare()
Load anchors from template:
StartAnchor = GetBlocInfo("Ancre_Holdings_START")
→ { UID="obj100", page=2, left=50, top=200, right=500, bottom=220 }
EndAnchor = GetBlocInfo("Ancre_Holdings_END")
→ { UID="obj101", page=4, left=50, top=700, right=500, bottom=720 }
```

```
Available height = 700 - 220 = 480 points
Available width = 500 - 50 = 450 points (unused for single column)
```

Step 4 — Process (Stage 1 — Get_Report):

SQL returns 150 rows. DReport built:

```
DSection[0]:
  DTable[0]:
    DRow[0]: level=0, cells=[{template="Header", names=["H1","H2","H3"], values=["Name","Wei
      page_break = true (header repeats on each page)
    DRow[1]: level=1, cells=[{template="DataRow", names=["D1_1","D1_2","D1_3"], values=["Sto
    DRow[2]: level=1, cells=[{template="DataRow", names=["D2_1","D2_2","D2_3"], values=["Sto
    ... (148 more data rows)
```

Step 4 — Process (Stage 2 — Check_Report):

```
For each DCell:
  cell.TElement = TElement_Helper.Evaluate_TElement(task, cell)
  "Header" → found in QXP_Project.Elements → TBox with height=20
  "DataRow" → found in QXP_Project.Elements → TBox with height=15
  All cells valid, none removed
```

Step 4 — Process (Stage 3 — Prepare_Report):

```
available_height = 480
cursor = DPoint(50, 220)
page = 1

Row[0] (header, page_break=true):
  → Stored as page_break_row, marked for removal
  → Geometry calculated: height=20, width=450

Row[1]: height=15, tableHeight=15
Row[2]: height=15, tableHeight=30
...
Row[32]: height=15, tableHeight=480 → equals available_height
Row[33]: height=15, tableHeight=495 → EXCEEDS available_height!
  → NEW PAGE: page=2
  → Clone header row (page_break_row) → insert at row[33] position
  → Reset tableHeight to header height (20) + row[33] height (15) = 35
  → cursor reset to startAnchor position

... continue until all rows placed

Result: 5 pages needed (page 1-5)
task.boxNames = ["H1","H2","H3","D1_1","D1_2","D1_3","D2_1",...] (all 450+ names)
report.info.nb_page = 5
```

Step 4 — Process (Stage 4 — Render_Boxes):

```
Create BlocPage(CREATE) for pages 0,1,2,3,4 (5 new pages)
Create BlocPage(REMOVE) for old pages 2,3,4 (3 old pages between anchors, offset by 5)
Move StartAnchor to page 0 (Pagination=true)
Move EndAnchor to page 4 (Pagination=true)
```

```

For each DCell → TElement_Helper.Get_Bloc(cell, task):
    Clone TBox, update name/value/position from cell info
    → BlocBox added to task.blocsModify

Total: 5 BlocPage(CREATE) + 3 BlocPage(REMOVE) + 2 anchor moves + ~450 BlocBox
All in task.blocsModify

```

Step 5 — Process_Steps:

```

RunTask.Get_Steps():
    Step[0]: UPDATE (DID value only - Direct_Call=false)
    Step[1]: PAGINATION (5 page creates + 3 page removes + 2 anchor moves - Direct_Call=true)
    Step[2]: MODIFY (450 BlocBox - Direct_Call=true)

```

```

Execute Step[1] (PAGINATION):
    Upload modifier XML to pool: PUT /addfile/R_2001/Modify_143025.xml
    Send: GET http://server:8080/R_2001/G_100_1.qxp?modify=file:Modify_143025.xml
    → QuarkXPress creates 5 pages, removes 3, moves anchors
    SaveAs: G_100_2.qxp

```

```

Execute Step[2] (MODIFY):
    Upload modifier XML: PUT /addfile/R_2001/Modify_143026.xml
    Send: GET http://server:8080/R_2001/G_100_2.qxp?modify=file:Modify_143026.xml
    → QuarkXPress creates 450 boxes with content
    SaveAs: G_100_3.qxp

```

Step 6 — Check (OVERFLOW DETECTED):

```

Refresh gabarit XML from G_100_3.qxp
GetOverflowBoxes() → ["D45_1", "D45_2", "D88_3"]

```

```

Task_Dynamique #20: controlOverflow = true
    boxNames contains "D45_1" → YES → add to overflowBoxes
    boxNames contains "D45_2" → YES → add to overflowBoxes
    boxNames contains "D88_3" → YES → add to overflowBoxes

    task.overflowBoxes = ["D45_1", "D45_2", "D88_3"]
    → Mark task for reprocessing

```

```

Set ALL other tasks' todo = false (except alwaysReprocess=true, like DID)
Task_Dynamique #20: todo = true, resetProcess()

```

```

RE-EXECUTE Step 4:
    Stage 2 - Check_Report:
        Cells containing "D45_1", "D45_2" → cell.overflow = true
        → Use template.srcNameOverflow instead of srcName
        → Overflow template has smaller font → more text fits

```

```

    Stage 3 - Prepare_Report:
        With overflow templates, rows are shorter
        Maybe only 4 pages needed now instead of 5

```

```

    Stage 4 - Render_Boxes:
        New blocs generated with overflow templates

```

```
RE-EXECUTE Step 5:
    Send new modifications to QuarkXPress Server

RETURN TO Step 6:
    GetOverflowBoxes() → [] (no more overflow)
    Continue to data collection
```

Example 3 — Task Exception (Row/Table Removal)

Scenario: Task_SQL #15 has an exception rule: if bloc "Amount_Q3" is empty, remove rows 2 and 4 from table "QuarterlyTable".

Step 4 — Process:

```
Task_SQL #15: Process()
    SQL returns:
        Row 1: NOM_BLOC="Amount_Q1", VALEUR_BLOC="1500.00" → BlocBox(UPDATE) in blocsUpdate
        Row 2: NOM_BLOC="Amount_Q2", VALEUR_BLOC="1800.00" → BlocBox(UPDATE) in blocsUpdate
        Row 3: NOM_BLOC="Amount_Q3", VALEUR_BLOC=NULL → BlocBox(UPDATE) with value=" " (nullS
        Row 4: NOM_BLOC="Amount_Q4", VALEUR_BLOC="2100.00" → BlocBox(UPDATE) in blocsUpdate

    Check_Task_Exception():
        Exception: name="Amount_Q3", tableName="QuarterlyTable",
                  indexLignes=[2,4], type=LINE

        Check: blocsUpdate contains "Amount_Q3"? YES
        Check: bloc.value == " " (nullString)? YES
        → processRemove = true

        Verify: gabarit.XML.GetNbLignes("QuarterlyTable") = 5 (≥ max index 4) ✓

        Create BlocLigne("QuarterlyTable_2", "QuarterlyTable", index=2), action=REMOVE
        Create BlocLigne("QuarterlyTable_4", "QuarterlyTable", index=4), action=REMOVE
        → Both added to task.blocsModify

Result:
    blocsUpdate: ["Amount_Q1"→"1 500.00", "Amount_Q2"→"1 800.00",
                  "Amount_Q3"→" ", "Amount_Q4"→"2 100.00"]
    blocsModify: ["QuarterlyTable_2"→REMOVE row 2, "QuarterlyTable_4"→REMOVE row 4]
```

Step 5 — How row removal is sent:

```
RunTask.Get_Steps():
    Step[0]: UPDATE (4 text values via SOAP)
    Step[1]: MODIFY (2 row removals via HTTP)

QXPS_Modifier for Step[1]:
    Add(BlocLigne "QuarterlyTable_2"):
        → AddGetTable("QuarterlyTable") → create Table wrapper
        → Add Ligne(index=2) to Table.Lignes

    Add(BlocLigne "QuarterlyTable_4"):
```

```
→ Table "QuarterlyTable" already exists
→ Add Ligne(index=4) to Table.Lignes
```

GetProject() → SDK output:

```
QXPSMSDK.Table {
  name: "QuarterlyTable",
  operation: "UPDATE",
  rows: [
    { index: 2, operation: "DELETE" },
    { index: 4, operation: "DELETE" }
  ]
}
```

Serialized XML:

```
<TABLE OPERATION="UPDATE">
  <ID NAME="QuarterlyTable"/>
  <ROW ROWCOUNT="2" OPERATION="DELETE"/>
  <ROW ROWCOUNT="4" OPERATION="DELETE"/>
</TABLE>
```

Example 4 — Mode Degradé (Large Template)

Scenario: Run ID 3001 has a template document of 15MB (exceeds 10MB threshold).

Step 2 – Load:

```
PrepareGabarit():
  Document data = byte[15728640] (15 MB)
  SizeLimitBeforeFailSoft = 10485760 (10 MB)
  15728640 > 10485760 → Mode_Degrade = true!
```

```
LoadInParams() → SKIPPED (Mode_Degrade)
LoadTasks() → SKIPPED (Mode_Degrade)
LoadTemplates() → SKIPPED (Mode_Degrade)
```

```
Step 3 – Prepare: SKIPPED (Mode_Degrade)
Step 4 – Process: SKIPPED (Mode_Degrade)
Step 5 – Process_Steps: SKIPPED (Mode_Degrade)
Step 6 – Check: SKIPPED (Mode_Degrade)
```

```
Step 7 – Render: RUNS (always runs!)
  Render unmodified template as PDF/QXP
  → Produces PDF of the template without any data filled in
```

Finally:

```
Add Critique error: "Run operating in Safe/Degraded Mode"
status = GENERATED (but with degraded flag)
Persist to DB with degraded indicator
```

17. Java Spring Boot Architecture

Package Structure


```

com.socgen.sgs.api.quark.engine/
├─ business/           → Gateway between service and DAO
├─ config/             → Spring configurations
├─ domain/             → Entities, value objects, ports
│  └─ bloc/           → BlocBase, BlocBox, BlocPage, etc.
│  └─ dynamic/        → DReport hierarchy, templates
│  └─ helper/         → DataTypeHelper
│  └─ port/           → Interface contracts (DocumentIdentityPort, FilePoolPort, etc.)
│  └─ task/           → TaskBase, TaskSQL, TaskDynamique, etc.
├─ dto/               → Data Transfer Objects
├─ enums/             → Shared enumerations
├─ infra/             → Infrastructure implementations
│  └─ api/            → REST controllers
│  └─ dao/            → Oracle data access (SimpleJdbcCall)
│  └─ interop/        → QuarkXPress HTTP/SOAP clients
├─ integration/       → SOAP client and WSDL-generated classes
├─ listener/          → RabbitMQ message listener
├─ mapper/            → Object-to-object mapping
└─ service/           → Application services and task strategies
   └─ task/           → TaskProcessStrategy implementations

```

Clean Architecture Rules (enforced by ArchUnit)

```

domain → CANNOT depend on service or infra
service → CANNOT depend on infra
domain & service → CANNOT use Spring Web, JPA, SQL, Jackson, Validation

```

Key Patterns

Aspect	Convention
Service interface	<code>..service.XxxService</code> — pure interface
Service impl	<code>..service.impl.XxxServiceImpl</code> — <code>@Service @Slf4j @RequiredArgsConstructor</code>
Business class	<code>..business.XxxBusiness</code> — <code>@Component</code> , single <code>execute()</code> method
DAO interface	<code>..infra.dao.XxxDao</code> — pure interface
DAO impl	<code>..infra.dao.impl.XxxDaoImpl</code> — <code>@Repository</code> , <code>SimpleJdbcCall</code>
Mapper	<code>..mapper.XxxMapper</code> — <code>@Component</code> , stateless
Domain	<code>..domain.Xxx</code> — Lombok , NO Spring annotations
Port	<code>..domain.port.XxxPort</code> — interface in domain , impl in infra
Task strategy	<code>TaskProcessStrategy<T extends TaskBase></code> — auto-discovered by Spring
Injection	Always <code>@RequiredArgsConstructor</code> with final fields
Testing	<code>@ExtendWith(MockitoExtension.class)</code> , <code>@InjectMocks</code> , <code>@Mock</code> , <code>@DisplayName</code>

18. Class Reference Index

Domain Classes

Class	Package	Purpose
Run	domain	Run state container
RunProperties	domain	Run configuration from DB
RunStatus	domain	Enum: TO_GENERATE, RUNNING, GENERATED, ERROR
InParam	domain	Single input parameter
DocumentDomain	domain	Document wrapper
DataNameValue	domain	Key-value pair for audit
TaskBase	domain.task	Abstract task base
TaskSQL	domain.task	SQL query task
TaskDynamique	domain.task	Dynamic report task
TaskSystem	domain.task	Simple value/box task
TaskDid	domain.task	Document Identity task
TaskCompartment	domain.task	Nested sub-document task
TaskDocument	domain.task	Image/PDF/document task
BlocBase	domain.bloc	Abstract bloc base
BlocBox	domain.bloc	Box modification
BlocPage	domain.bloc	Page create/remove
BlocTable	domain.bloc	Table removal
BlocLigne	domain.bloc	Row removal
BlocGroup	domain.bloc	Multi-box group
DReport	domain.dynamic.report	Dynamic report root
DSection	domain.dynamic.report	Report section
DTable	domain.dynamic.report	Relative positioning table
DRow	domain.dynamic.report	Table row
DCell	domain.dynamic.report	Data cell
Template	domain.dynamic.template	Template configuration

Service Classes

Class	Package	Purpose
ProcessRunService	service	Pipeline orchestrator interface
LoadTasksService	service	Step 2 task loading
ProcessTasksService	service	Step 4 three-phase loop
ProcessStepsService	service	Step 5 QXP server communication
OverflowCheckService	service	Step 6 overflow detection
RenderService	service	Step 7 document rendering
TaskProcessStrategy	service.task	Per-task-type processing
TaskPostProcessStrategy	service.task	Per-task-type post-processing

Infrastructure Classes

Class	Package	Purpose
QxpsHttpClient	infra.interop.qxps.client	HTTP calls to QXP Server
QxpsRequestBuilder	infra.interop.qxps.request	URL construction from messages
FilePoolService	infra.interop.qxps.pool	File pool operations
EngineSoapClient	integration.soap.client	SOAP calls to QXP Manager
All DAO implementations	infra.dao.impl	Oracle data access

End of Documentation